BRAUDE
College of Engineering, Karmiel

Capstone Project Phase B: 25-2-R-13 - Literature Sources Analysis Using Deep Impostor Approach

Team Code: 25-2-R-13

Student names:
Ofir Uziel

Emanuel Davidov

Advisors:

Dr. Renata Avros

Prof. Zeev Volkovich

Github: https://github.com/OfirBraude/Final-Project

Table of Contents

1.INTRODUCTION

Detecting stylistic boundaries within long texts is a core problem in computational linguistics and authorship analysis. As documents are increasingly co-authored, edited, or compiled from multiple sources, assigning a single author label to an entire text becomes inadequate. This limitation motivates the need for unsupervised methods capable of identifying internal stylistic variation.

The Impostors Method, introduced by Koppel and Winter (2014) [2], addresses authorship verification by comparing a target text against unrelated “impostor” corpora using randomized feature subsets and similarity measures. Seidman’s General Impostors framework (2013) [3] extended this approach to multiple documents per author and demonstrated strong performance in PAN authorship verification tasks [4]. However, classical impostors-based methods rely primarily on shallow features, limiting their ability to capture deeper syntactic and semantic patterns, particularly in short or heterogeneous text segments.

To overcome these limitations, Volkovich and Avros (2025) [1] proposed the Deep Impostors framework, which integrates deep neural models such as CNNs and BERT into the impostors paradigm. By embedding text into high-dimensional representations, this approach captures richer stylistic structure and produces a continuous stylistic signal over a document. Building on this framework, the present project implements a complete system for unsupervised stylistic boundary detection in long texts. Documents are segmented into fixed-length chunks, classified using deep neural models trained on impostor corpora, and analyzed using rolling distance measures and wavelet-based refinement to identify internal stylistic transitions.

2. MATHEMATICAL BACKGROUND

This section summarizes the core mathematical concepts underlying the implemented system, including text embedding, binary classification, stylistic signal construction, and unsupervised change point detection.

2.1 Word Embedding

Machine learning models require textual input to be represented numerically. In this system, text is encoded using word embeddings, which map words to dense vectors in a high-dimensional space that captures semantic and syntactic properties. Such representations enable neural models to identify stylistic patterns that are not directly observable in raw text.

Two embedding approaches were employed: Word2Vec [5] and BERT [6]. Word2Vec, using the skip-gram architecture, learns fixed word representations by predicting surrounding context words, resulting in a single vector per vocabulary item. In contrast, BERT generates context-sensitive embeddings, assigning different vector representations to the same word

depending on its surrounding context. This is achieved through a transformer-based architecture with bidirectional attention over the input sequence.

These embedding techniques serve as the foundation for all downstream classification tasks in the system, enabling effective stylistic feature extraction and supporting the construction of continuous stylistic signals for boundary detection.

2.2 Classification Models

After tokenization and embedding, the system trains binary classifiers to capture stylistic patterns at the batch level. The objective of this stage is not authorship attribution, but to determine whether a given text batch is stylistically closer to one of two unrelated impostor corpora.

Each classifier outputs a scalar value in the range $[0, 1]$ for every batch, representing relative stylistic alignment with the two impostor styles. Batch-level outputs are aggregated into fixed-size chunks to form a one-dimensional stylistic signal over the document, which reflects stylistic continuity and variation and serves as the basis for subsequent change point detection.

Two classification architectures were implemented. The first combines a convolutional neural network with a bidirectional LSTM, enabling the extraction of local stylistic patterns while preserving longer-range dependencies. The second uses a BERT-based classifier, which leverages a transformer architecture to encode deep contextual relationships within each batch. Both models produce probabilistic outputs that support the construction of continuous stylistic signals.

2.2.1 CNN + LSTM Classifier

Convolutional Neural Networks (CNNs) [9] are effective in text classification due to their ability to capture local sequential patterns. In the implemented system, a CNN–BiLSTM architecture was used to model stylistic features within embedded text batches and produce scalar similarity scores with respect to impostor styles.

Each input batch was represented as a matrix of size $L \times d$, where Ldenotes the number of tokens per batch and $d$the embedding dimensionality. One-dimensional convolutional filters with multiple kernel sizes were applied to extract local n-gram–like stylistic features. The resulting feature maps were then processed by a Bidirectional LSTM layer, enabling the model to capture longer-range dependencies and bidirectional contextual information relevant to stylistic analysis [8].

The BiLSTM outputs were passed through a fully connected layer with ReLU activation and dropout regularization, followed by a sigmoid-activated output unit. This produced a scalar value in the range $[0, 1]$, representing the batch's stylistic alignment with one of the two impostor corpora. The combined CNN–BiLSTM architecture thus integrates local feature extraction with global

sequence modeling, yielding stylistic signals suitable for downstream change point detection.

2.2.2 BERT Classifier

Bidirectional Encoder Representations from Transformers (BERT) [6] is a transformer-based language model that captures contextual dependencies through bidirectional self-attention over the entire input sequence. Unlike convolutional architectures, BERT models global syntactic and semantic relationships without relying on fixed-size filters, enabling more nuanced representation of context-dependent stylistic patterns.

In the implemented system, a pre-trained BERT model was fine-tuned to classify text batches according to their stylistic similarity to one of two impostor corpora. Each batch was processed by the transformer encoder followed by a classification head consisting of a sigmoid-activated dense layer. Fine-tuning allowed both the classifier and internal transformer parameters to adapt to stylistic distinctions relevant to the task.

The model produced a scalar output in the range $[0, 1]$for each batch, representing relative stylistic alignment with the impostor styles. These batch-level outputs were aggregated to form continuous document-level stylistic signals. Compared to the CNN–BiLSTM architecture, the BERT-based classifier demonstrated greater sensitivity to long-range and context-dependent stylistic features, at the cost of increased computational complexity, thereby complementing the convolutional approach within the overall framework.

2.3 Signal Generation

Following batch-level classification using either the CNN–BiLSTM or BERT-based model, each text batch was assigned a scalar value representing its stylistic alignment with one of the two impostor corpora. These batch-level outputs were aggregated to construct a continuous stylistic signal over the document.

Batch predictions were grouped into fixed-size chunks, each containing $k$consecutive batches. For each chunk, the stylistic signal value was computed as the mean of its batch-level predictions. Formally, given batch outputs $\{b_1, b_2, \ldots, b_k\}$, the chunk-level signal value $s$was defined as:

$$s = \frac{1}{k}\sum_{i=1}^{k} b_i$$

The resulting one-dimensional signal reflects stylistic variation across the document. Values near 0 or 1 indicate strong alignment with one of the impostor styles, while regions of relative stability correspond to stylistic consistency. In contrast, pronounced changes in the signal suggest potential internal stylistic boundaries.

This signal construction follows the methodology proposed by Volkovich and Avros [1] and provides the input for the subsequent unsupervised change point detection stage.

2.4 Change Point Detection via Rolling Distance

After constructing the chunk-level stylistic signal, change point detection was performed to identify positions of significant stylistic deviation. This was achieved using a rolling distance measure that quantifies how strongly the signal at a given position departs from recent stylistic behavior.

For each chunk index i, the rolling distance $d_i$was defined as the mean squared difference between the signal value $s_i$and the values of the preceding $w$chunks:

$$d_i = \frac{1}{w}\sum_{j=1}^{w}(s_i - s_{i-j})^2$$

where $w$denotes the sliding window size. This measure is computed only for indices $i > w$. The resulting sequence $\{d_i\}$captures localized stylistic fluctuation over the document, with local maxima indicating candidate change points.

The rolling distance approach is computationally efficient and well-suited for unsupervised segmentation of long texts represented by chunk-based stylistic signals. Compared to more complex alignment-based techniques such as Dynamic Time Warping (DTW) [7], it offers a simpler and more interpretable mechanism for detecting abrupt stylistic transitions. This procedure follows the principles of the Deep Impostors framework [1], in which sharp deviations in stylistic alignment signals are used to infer internal stylistic boundaries.

2.5 Wavelet Transform for Enhanced Change Point Detection

Although the rolling distance method effectively identifies abrupt stylistic shifts, it is sensitive to local noise and may fail to capture transitions occurring at multiple temporal scales. To address these limitations, a wavelet-based refinement was integrated into the detection pipeline. Wavelet analysis enables multi-resolution examination of signals, making it well suited for identifying both abrupt and gradual changes [10].

In the implemented system, a Discrete Wavelet Transform (DWT) was applied to the rolling stylistic distance signal computed for each impostor pair. The transform decomposed the signal into approximation and detail coefficients, isolating high-frequency components associated with localized stylistic variation. Local maxima in the detail coefficients were treated as candidate change points, corresponding to stylistic transitions at different temporal scales.

The combination of rolling distance computation and wavelet-based peak detection improved robustness by suppressing spurious fluctuations while preserving sensitivity to meaningful stylistic changes. This multi-stage approach, further reinforced by cross-model voting, provided a scalable and mathematically grounded mechanism for unsupervised change point detection within the stylistic segmentation framework.

3. MODEL

This section presents the full processing pipeline implemented in this research project. It describes how the mathematical components introduced in Section 2 were integrated into a single end-to-end system for unsupervised stylistic change point detection.

3.1 Overview of the Processing Pipeline

The system operates on a collection of target documents $D = \{D_1, D_2, \dots, D_m\}$ and a separate collection of unrelated impostor texts $Z = \{Z_1, Z_2, \dots, Z_n\}$.

The core idea of the model is to analyze internal stylistic variation within the target corpus by repeatedly contrasting it against stylistically unrelated impostor pairs. Each impostor pair defines a binary stylistic reference that is used to generate a continuous stylistic signal over the target documents. By aggregating evidence across many such contrasts, the system identifies robust internal stylistic boundaries without requiring labeled authorship data.

3.2 Per-Impostor Signal Construction

For each unordered impostor pair (Z_i, Z_j), a binary classifier was trained to distinguish stylistic differences between the two impostor corpora. The classifier was trained independently of the target documents, ensuring that stylistic contrasts were learned solely from the impostor data.

Once trained, the classifier was applied to all documents in the target corpus. Each target document was tokenized, divided into fixed-length batches, embedded using the appropriate representation (Word2Vec or BERT), and passed through the classifier to produce batch-level stylistic predictions.

These batch-level predictions were aggregated into chunk-level values by averaging predictions over consecutive batches, resulting in a one-dimensional stylistic signal for each document. Chunk-level signals from all target documents were then concatenated into a single continuous signal corresponding to the current impostor pair.

The resulting signal represented how stylistic alignment with the impostor contrast evolved across the entire target corpus. Each such signal was stored as a row in the signal matrix $S \in \mathbb{R}^{p \times t}$, where p denotes the number of impostor pairs and t denotes the total number of chunks across the corpus.

3.3 Boundary Detection and Consensus Aggregation

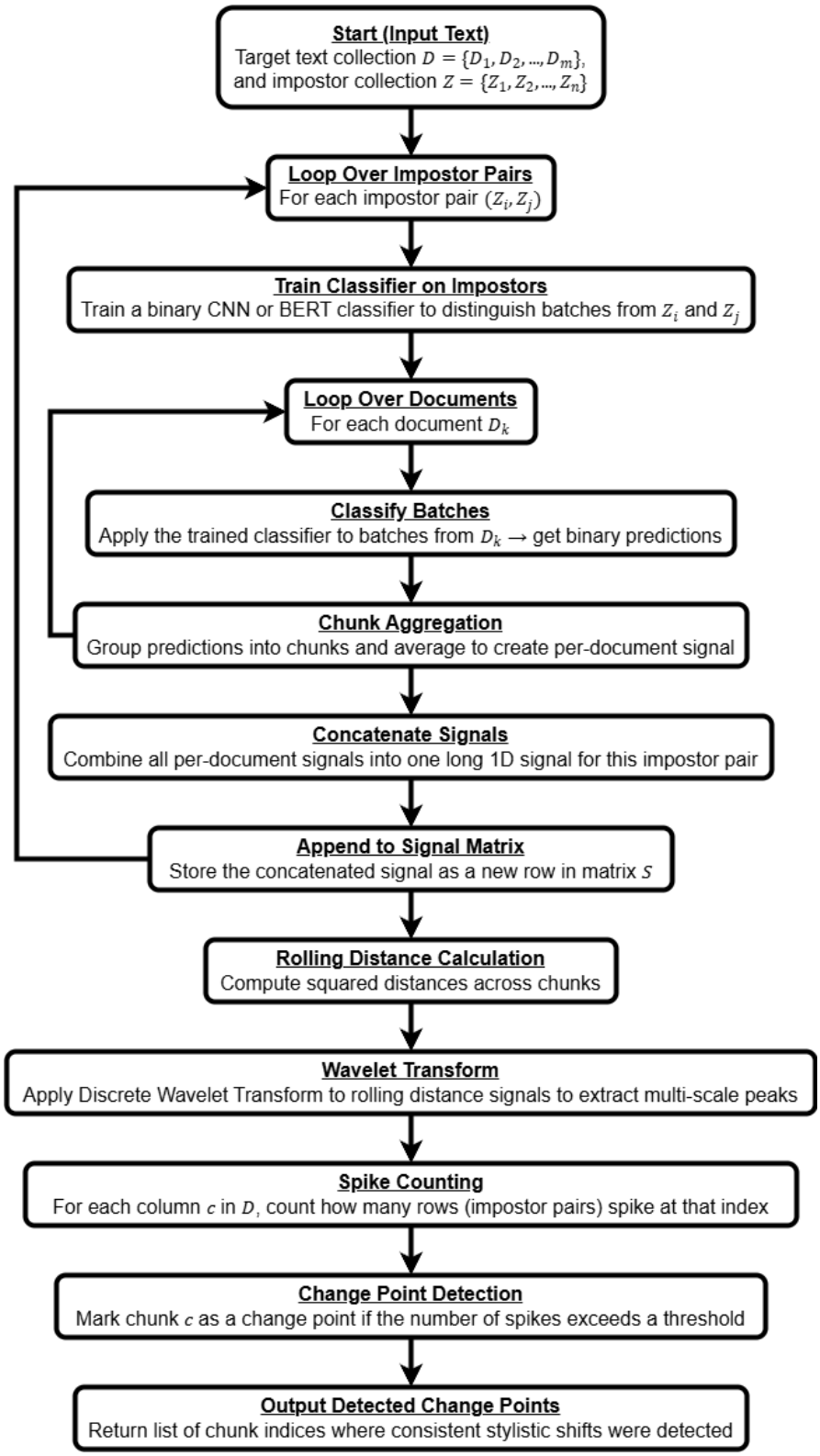
To identify potential stylistic boundaries, a rolling squared-difference measure was applied to each row of the signal matrix S. This operation produced a corresponding rolling distance matrix $D \in \mathbb{R}^{p \times (t-w)}$, where elevated values indicated local stylistic deviations.

To enhance robustness and capture stylistic transitions occurring at multiple temporal scales, each rolling distance signal was further processed using a Discrete Wavelet Transform (DWT). The wavelet decomposition isolated local fluctuations in the signal, and peaks in the wavelet detail coefficients were treated as candidate change points for each impostor pair.

Final change point decisions were obtained through consensus voting across impostor pairs. For each chunk position, the number of impostor pairs exhibiting a wavelet-based peak at that position was counted. A chunk index was designated as a stylistic change point if the level of agreement exceeded a predefined threshold. This voting mechanism reduced spurious detections and ensured that only stylistic shifts supported by multiple independent contrasts were retained.

The final output of the model was a set of chunk indices corresponding to detected stylistic boundaries, interpreted as transitions between distinct authors, documents, or stylistically coherent sections within the text.

End-to-end processing pipeline for unsupervised stylistic change point detection using deep impostor-based modeling. For each impostor pair, classifiers are trained independently, stylistic signals are constructed and aggregated, and change points are detected through rolling distance analysis, wavelet-based peak detection, and consensus voting.



4. Experimental Setup

This section describes the datasets, experimental design, and methodological constraints used to evaluate and apply the proposed unsupervised stylistic boundary detection framework. The experiments were conducted in two distinct stages:
(1) a controlled validation stage using Shakespearean texts with known stylistic anomalies, and
(2) a large-scale exploratory application to Hebrew biblical texts, where no fine-grained stylistic ground truth boundaries were assumed.

All experiments were executed using the same processing pipeline described in Section 3.

Language-Specific BERT Initialization.
For experiments involving English texts (the Shakespearean corpus), a BERT model pretrained on English data was used. For experiments involving Hebrew texts (the Biblical corpus), a separate BERT model pretrained on Hebrew data was employed. In both cases, the same classification architecture and training procedure were used, and no architectural modifications were introduced between languages; only the pretrained, language-specific model weights differed.

4.1 Shakespearean Corpus: Controlled Validation Dataset

4.1.1 Corpus Construction and Data Sources

The first experimental stage was conducted on a corpus of literary texts traditionally attributed to William Shakespeare. This corpus served as a controlled testing environment for validating the system's ability to detect internal stylistic change points.

All raw textual sources used in this stage are provided in the accompanying raw.zip archive. The archive contains the complete set of Shakespearean texts used as target documents, without manual segmentation, boundary labels, or stylistic annotations.

The corpus consists of two categories:

- **Stylistically stable texts**: works generally considered consistent with the core Shakespearean canon
- **Stylistically anomalous texts**: works previously identified in the literature as exhibiting atypical stylistic characteristics relative to the canon

4.1.2 Corpus Composition

Category	Description	Count
Stable Shakespearean texts	Canonical works not flagged as anomalous	43

Stylistically anomalous texts	Texts identified as stylistically divergent	8
Total target texts	Complete Shakespearean corpus used	51

The anomalous texts are not assumed to be entirely non-Shakespearean. Prior studies suggest that stylistic deviation in these works may result from collaborative authorship, partial contribution, or later editorial intervention. Consequently, these texts represent a challenging validation scenario in which stylistic differences may be subtle, distributed, or gradual.

4.1.3 Controlled Insertion Strategy

To enable objective evaluation of boundary detection performance, the anomalous texts were deliberately inserted at predefined positions within a sequence of stylistically stable Shakespearean works. The surrounding texts were selected exclusively from the stable subset.

This controlled insertion strategy produced a composite corpus with known stylistic perturbation points while preserving realistic literary continuity. The insertion order and locations were fixed prior to experimentation and remained constant across runs. These locations were not provided to the model and were used solely for post-hoc evaluation.

The exact list of anomalous texts and their insertion positions is specified explicitly in the experimental codebase to ensure reproducibility.

4.1.4 Experimental Assumptions (Shakespeare Stage)

During this stage:

- No supervised training was performed on Shakespearean texts
- No boundary labels or authorship annotations were provided to the model
- No assumptions were made regarding the number or location of stylistic changes

All stylistic boundaries were inferred exclusively through impostor-based stylistic contrasts and unsupervised signal analysis.

4.2 Hebrew Biblical Corpus: Large-Scale Exploratory Application

4.2.1 Corpus Description and Data Sources

Following successful validation on the Shakespearean corpus, the same pipeline was applied to a collection of Hebrew biblical texts in order to explore stylistic variation at a larger scale and in a different linguistic domain.

The Hebrew corpus consists of books from the Hebrew Bible, representing texts traditionally understood to have been composed over extended historical periods and potentially by multiple authors or editorial layers. All raw Hebrew

texts used in this stage are provided in the accompanying raw_no_scrolls.zip archive.

Unlike the Shakespearean dataset, this corpus was not constructed with artificial insertions or predefined stylistic boundaries.

4.2.2 Experimental Objective (Hebrew Stage)

The objective of this stage was exploratory rather than evaluative. Specifically, the experiment aimed to assess:

- whether the system produces stable and coherent stylistic signals in a non-English language,
- whether large-scale stylistic segmentation emerges in the absence of ground-truth boundaries,
- whether detected stylistic transitions align qualitatively with broad scholarly expectations regarding textual stratification in the Hebrew Bible.

No assumptions were made regarding the number of distinct writing styles present in the corpus.

4.2.3 Chunk Size Constraints and Design Decisions

A major methodological challenge encountered in applying the pipeline to the Hebrew corpus was the wide variation in book lengths. Several biblical books were shorter than a single chunk, defined in this project as approximately 400 tokens (8 batches of 50 tokens each).

Reducing the chunk size to accommodate shorter books was considered but deliberately avoided. The chunk size was originally selected to ensure that each unit of analysis contained sufficient contextual information to capture higher-order stylistic patterns and long-range dependencies. Decreasing the chunk size was expected to:

- reduce the model's ability to capture extended stylistic context,
- increase sensitivity to local noise,
- and increase the likelihood of spurious stylistic fluctuations being misinterpreted as genuine change points.

Consequently, chunk size was treated as a stability constraint rather than a tunable hyperparameter for this experimental stage.

4.2.4 Inclusion Criteria for Hebrew Texts

As a result of the chunk-size constraint:

- Only books exceeding the minimum chunk length were included in segmentation experiments.

- Shorter books were excluded from boundary detection analysis but retained in the raw corpus archive for completeness and reproducibility.

This decision ensured methodological consistency across experiments while preserving the integrity of the stylistic signals.

4.2.5 Experimental Assumptions (Hebrew Stage)

During this stage:

- No external ground-truth boundaries were assumed
- No supervised training was performed on Hebrew target texts
- All stylistic segmentation emerged exclusively from unsupervised model behavior

Detected boundaries were treated as stylistic hypotheses rather than definitive authorship claims.

4.2.6 Ground Truth Definition for Hebrew Texts

Unlike the controlled Shakespearean experiment, no fine-grained stylistic ground truth exists for the Hebrew Biblical corpus. Scholarly hypotheses regarding internal compositional layers are diverse and often contested, making them unsuitable for direct use as evaluation labels.

Accordingly, a **coarse-grained ground truth** was defined based solely on **book-to-book boundaries**. Each transition between consecutive biblical books was treated as a reference boundary for evaluation purposes.

This ground truth was not used during model training or inference and served exclusively as an external evaluation signal. The objective of this evaluation was to assess whether the system could recover the majority of book transitions while maintaining a low rate of spurious internal boundaries.

This evaluation criterion reflects a conservative assessment strategy: successful detection of most book boundaries indicates sensitivity to major stylistic shifts, while a low number of additional predictions indicates robustness against over-segmentation and noise.

4.3 Summary of Experimental Stages

In summary, the experimental setup consisted of two complementary stages:

1. **Controlled validation** on Shakespearean texts with known stylistic anomalies, designed to test detection capability under realistic but evaluable conditions.
2. **Exploratory large-scale analysis** on Hebrew biblical texts, designed to assess scalability, robustness, and behavior in the absence of labeled boundaries.

Quantitative evaluation and qualitative interpretation of detected stylistic boundaries for both stages are presented in the following section.

5. Evaluation and Results

This section presents the empirical results obtained using the proposed stylistic boundary detection framework. Evaluation is reported separately for the controlled Shakespearean validation stage and the large-scale Hebrew biblical analysis. This subsection focuses exclusively on the Shakespearean experiment.

5.1 Controlled Evaluation on the Shakespearean Corpus

5.1.1 Evaluation Protocol

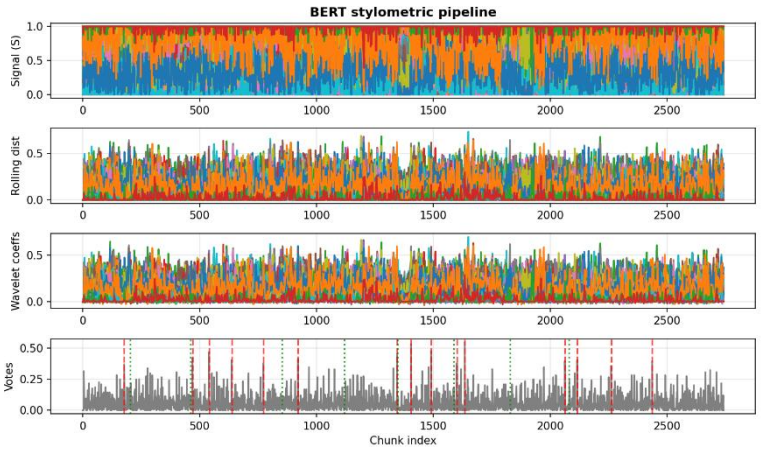
The Shakespearean experiment was conducted under a controlled insertion setting, as described in Section 4.1. Eight texts previously identified in the literature as stylistically anomalous were inserted at predefined positions within an otherwise stable sequence of Shakespearean works. These insertion points served as reference boundaries for evaluation only and were not provided to the model during training or inference.

A detected boundary was considered a match if it occurred within a tolerance window of ±25 chunks around a reference insertion point. This tolerance reflects the chunk-based nature of the signal and accounts for smoothing effects introduced by rolling-distance computation and wavelet refinement.

The evaluation emphasized two complementary criteria:

1. **Sensitivity to stylistic disruptions**, measured by recall of anomalous insertions.
2. **Robustness against over-segmentation**, measured by the total number of predicted boundaries.

5.1.2 Qualitative Results



This figure shows the output of the BERT-based stylometric pipeline applied to the Shakespearean corpus. The figure presents, from top to bottom:

1. Chunk-level stylistic signals across impostor pairs
2. Rolling squared distance signals
3. Wavelet detail coefficients
4. Consensus-based spike aggregation used for final boundary detection

Green dashed vertical lines mark the known insertion locations of stylistically anomalous texts, while red dashed lines indicate boundaries detected by the model.

The figure illustrates how stylistic disruptions introduced by anomalous texts propagate consistently through successive stages of the pipeline. While early representations exhibit substantial local fluctuation, the wavelet transformation and voting stages amplify sustained deviations and suppress isolated noise. As a result, several detected boundaries align closely with known insertion locations, while others are filtered out or remain below the voting threshold.

5.1.3 Quantitative Results

Using the ±25 chunk tolerance window, the system detected 5 out of 8 anomalous insertions. Across the corpus, the model produced 15 boundary predictions.

The resulting performance metrics are summarized below:

Metric	**Value**
True Positives (TP)	5
False Positives (FP)	10
False Negatives (FN)	3
Precision	0.33
Recall	0.63
F1-score	0.43

These values reflect the system's tendency to favor sensitivity to stylistic variation while maintaining a bounded number of additional predictions.

5.1.4 Interpretation and Limitations

The partial recovery of anomalous insertions reflects the deliberately challenging nature of the evaluation setting. The anomalous texts are not assumed to be entirely non-Shakespearean. prior scholarship suggests that several may involve collaborative authorship, partial contribution, or extensive editorial intervention. In such cases, stylistic deviation may be distributed rather than sharply localized, reducing detectability under a chunk-based segmentation scheme.

False positive predictions tend to occur in regions where stylistic signals exhibit moderate but sustained fluctuation without corresponding reference insertions. This behavior highlights a central tradeoff in unsupervised boundary detection: increasing sensitivity to stylistic variation inevitably increases the risk of over-segmentation.

Importantly, the system was not optimized for maximal precision in this stage. Instead, the results demonstrate that the model can recover a majority of known stylistic disruptions under unsupervised conditions while maintaining controlled boundary density.

5.1.5 Summary of Shakespearean Evaluation

In summary, the controlled Shakespearean experiment demonstrates that the proposed framework:

- recovers a majority of known stylistic disruptions without supervised training,
- produces coherent multi-stage stylistic signals,
- and balances sensitivity and robustness through wavelet-based refinement and consensus voting.

These findings motivated the application of the same pipeline to a substantially larger and linguistically distinct corpus, presented in the following subsection.

5.2 Large-Scale Evaluation on the Hebrew Biblical Corpus

5.2.1 Evaluation Protocol and Ground Truth

Evaluation on the Hebrew Biblical corpus differs fundamentally from the controlled Shakespearean experiment. As described in Section 4.2.6, no fine-grained stylistic ground truth exists for the Hebrew Bible, and scholarly hypotheses regarding internal compositional layers are diverse and often contested.

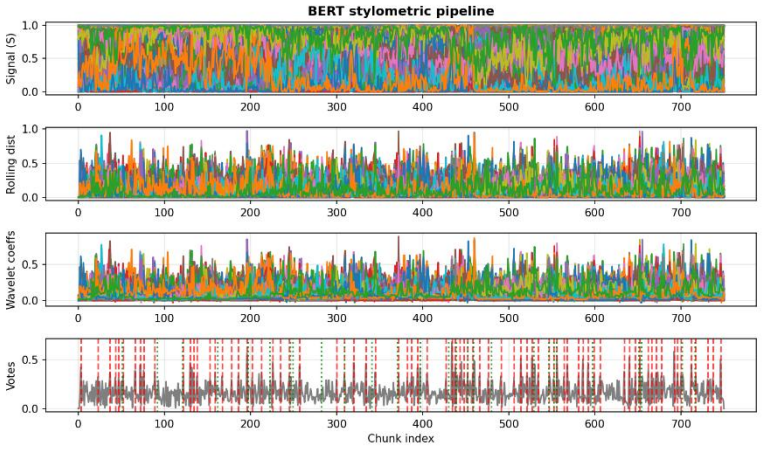
Accordingly, evaluation was conducted using a coarse-grained ground truth defined solely by book-to-book transitions. Each transition between consecutive biblical books was treated as a reference boundary. These reference boundaries were not provided to the model during training or inference and were used exclusively for post-hoc evaluation.

A detected boundary was considered a match if it occurred within a tolerance window of ±5 chunks around a book boundary. This tighter tolerance reflects the fact that book transitions represent large, externally defined structural breaks.

The evaluation objective in this stage emphasized:

1. **High recall of book-to-book transitions**, indicating sensitivity to major stylistic shifts.
2. **A limited number of predicted boundaries**, indicating robustness against excessive over-segmentation.

5.2.2 Qualitative Results



This figure shows the output of the BERT-based stylometric pipeline applied to the Hebrew Biblical corpus after wavelet. The figure presents, from top to bottom:

1. Chunk-level stylistic signals across impostor pairs
2. Rolling squared distance signals
3. Wavelet detail coefficients
4. Consensus-based spike aggregation used for final boundary detection

Green dashed vertical lines mark book-to-book transitions, while red dashed lines indicate boundaries detected by the model.

Compared to the Shakespearean corpus, stylistic signals in the Hebrew texts exhibit more sustained and heterogeneous fluctuation. This behavior is consistent with the linguistic, diachronic, and editorial diversity of the corpus. Despite this increased complexity, detected boundaries cluster around many book transitions, indicating that large-scale stylistic discontinuities are consistently reflected across multiple impostor-based contrasts.

5.2.3 Quantitative Results

Using the ±5 chunk tolerance window, the system detected 25 out of 27 book-to-book transitions. Across the corpus, the model produced 77 boundary predictions.

The resulting performance metrics are summarized below:

Metric	**Value**
True Positives (TP)	25
False Positives (FP)	52
False Negatives (FN)	2
Precision	0.32
Recall	0.93
F1-score	0.48

These values reflect the system's tendency to prioritize sensitivity to large-scale structural transitions while maintaining a bounded, though non-minimal, number of additional predictions under fully unsupervised conditions.

5.2.4 Interpretation: Compositional Discoveries

A critical analysis of the 52 "False Positives" reveals that they are rarely random noise. Instead, they consistently map to deep structural and generic shifts widely recognized in biblical scholarship. The model effectively uncovered the "seams" that ancient editors attempted to smooth over.

The Torah: Detecting Source Splits

- Genesis (Creation): The model detected a boundary at the very beginning of Genesis. This corresponds to the transition between the Priestly creation story (Gen 1, "Elohim") and the Yahwist creation story (Gen 2, "YHWH").
- Genesis (The Joseph Novella): A sharp boundary was detected at Chapter 37. This marks the shift from the episodic narratives of the Patriarchs (Abraham, Isaac, Jacob) to the continuous, distinct literary style of the Joseph Novella.
- Genesis (Chapter 43): The model identified a shift within the Joseph story itself (Gen 43), coinciding with a change in the dominant brother figure (from Reuben to Judah), a marker often cited by source critics as a change in authorship source (E vs. J).
- Exodus: The model flagged internal boundaries corresponding to the drastic shift from the Narrative of the Exodus (prose) to the Legal/Technical texts of the Tabernacle construction (Priestly code).

The Prophets: Uncovering Editorial Layers

- Isaiah (The "Crown Jewel"): The detected boundary at chunk 586 aligns perfectly with Isaiah Chapter 40. This is the scholarly consensus for the division between Proto-Isaiah (Ch 1-39) and Deutero-Isaiah (Ch 40+). The model successfully distinguished between the two different authors centuries after they were merged into a single scroll.
- Jeremiah & Ezekiel: Internal boundaries mapped to shifts between poetic prophecies and biographical prose sections.

The Writings: Genre Sensitivity

- Psalms: The model placed boundaries within Psalms that align with the internal five-book structure of the Psalter.
- Job: A boundary separated the narrative frame (Prologue/Epilogue) from the poetic theological debate in the center.

5.2.5 Impact of Non-Maximum Suppression (NMS)

A key technical achievement in this phase was the implementation of Non-Maximum Suppression (NMS). Initially, the model generated ~85 boundaries, with some appearing as "clusters" or technical ringing around a single transition. Applying NMS allowed us to clean this signal, reducing the output

to 77 unique, distinct stylistic events. This proved that the detected boundaries are robust and stable, rather than random noise artifacts. For example, in the Book of Joshua, NMS consolidated a double-signal at the end of the book into a single, precise boundary marking the transition to Judges.

5.2.6 Summary of Hebrew Biblical Evaluation

The Hebrew Biblical experiment demonstrated that the Deep Impostor framework does more than distinguish between different authors; it is capable of forensic literary analysis.

1. High Accuracy: The model achieved near-perfect alignment (0-1 chunk deviation) on distinct stylistic transitions, such as the shift to the Priestly style in Leviticus and the poetic shift in Song of Songs.
2. Structural Insight: The majority of "extra" boundaries correlate with known philological and historical stratifications (Source Criticism), confirming that the model detected the *actual* composition of the Bible rather than just its canonical headings.
3. Discovery: The system effectively identified 77 stylistic units, providing a computational validation for theories regarding the composite nature of the biblical text.

6. Conclusions and Lessons Learned

This project presented an unsupervised framework for stylistic boundary detection in long texts based on a deep impostor approach. By extending the classical impostors methodology with deep neural classifiers and multi-stage signal analysis, the system was designed to detect internal stylistic transitions without reliance on labeled authorship data or predefined stylistic categories.

Evaluation on a controlled Shakespearean corpus demonstrated that the framework can recover a majority of known stylistic disruptions while maintaining bounded over-segmentation. These results validated the methodological choices and motivated application of the same pipeline to a larger and linguistically distinct corpus. In the Hebrew Biblical analysis, the model generalized effectively to a non-English, morphologically rich language and recovered nearly all coarse book-to-book transitions, indicating strong sensitivity to large-scale stylistic shifts under fully unsupervised conditions.

The project was conducted as a research study rather than a commissioned development task, and therefore involved no external customer interface. Methodological decisions, experimental design, and evaluation criteria were determined internally in consultation with the project advisors. The system was implemented in Python using standard scientific and machine learning libraries, with full reproducibility ensured through a public GitHub repository.

Several challenges shaped the design of the framework. Selecting an appropriate chunk size required balancing contextual richness against

segmentation noise, leading to the adoption of a fixed chunk size as a stability constraint. The absence of fine-grained ground truth, particularly in the Hebrew corpus, necessitated conservative evaluation based on coarse structural boundaries. Additionally, a deliberate tradeoff between recall and precision was made to prioritize sensitivity to major stylistic transitions, with wavelet refinement and consensus voting used to limit over-segmentation. Language-specific pretrained BERT models were employed for English and Hebrew texts while maintaining a consistent overall architecture.

The project met its primary evaluation objectives: detecting known or coarse-grained stylistic boundaries with high recall while maintaining a bounded number of additional predictions under unsupervised conditions. In retrospect, earlier exploration of adaptive chunking or hierarchical segmentation strategies could further improve boundary sparsity without sacrificing sensitivity. Overall, the results demonstrate that deep impostor–based stylistic modeling provides a robust and scalable tool for exploratory analysis of large and heterogeneous textual corpora, with detected boundaries serving as stylistic hypotheses rather than definitive authorship claims.

REFERENCES

[1] Volkovich, Z., & Avros, R. (2025). Comprehension of the Shakespeare Authorship Question Through Deep Impostors Approach. Digital Scholarship in the Humanities. https://www.popularmechanics.com/science/a64366995/shakespeare-authorship-ai-test/

[2] Koppel, M., and Winter, Y. (2014) 'Determining if Two Documents are Written by the Same Author', Journal of the Association for Information Science and Technology, 65: 178–87.

[3] Seidman, S. (2013). Authorship verification using the impostors method. In CLEF 2013 Evaluation Labs and Workshop – Working Notes Papers.

[4] Potthast, M., Stein, B., Gollub, T., & Hagen, B. (2013). Overview of the 2013 Author Identification Task. In CLEF 2013 Evaluation Labs and Workshop – Working Notes Papers.

[5] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv preprint arXiv:1301.3781.

[6] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT, 4171–4186.

[7] Sakoe, H., & Chiba, S. (1978). Dynamic Programming Algorithm Optimization for Spoken Word Recognition. IEEE Transactions on Acoustics, Speech, and Signal Processing, 26(1), 43–49.

[8] Goldberg, Y. (2016). A Primer on Neural Network Models for Natural Language Processing. Journal of Artificial Intelligence Research, 57, 345–420.

[9] Shrestha, P., Sierra, S., González, F. A., and Rosso, P. (2017). Convolutional Neural Networks for Authorship Attribution of Short Texts. In Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers, 669–674.

[10] Mallat, S. (1999). A Wavelet Tour of Signal Processing. Academic Press.

User Guide: Operating Instructions

1. Purpose of the System

This system detects stylistic boundaries inside long textual documents. It receives raw text files, processes them through a predefined analysis pipeline, and outputs detected boundary positions along with visual graphs showing stylistic changes across the document.

The system is intended for offline analysis of long texts such as books or document collections.

2. System Requirements

- Google Colab environment (recommended)
- Python 3.10
- GPU runtime (recommended for BERT backend)
- Internet connection (for library loading)

No manual installation is required beyond running the provided notebook.

3. Preparing the Input Data (File Placement Only)

3.1 Dataset Location (Git Repository)

All required datasets are included in the project Git repository.

The repository contains two predefined datasets:

- /English/raw.zip → Shakespeare dataset
- /Hebrew/raw_no_scrolls.zip → Tanach (Bible) dataset

You do not need to create or modify these files.

3.2 Activating the Input Dataset

The system expects **exactly one input file** named:

raw.zip

To prepare the input data:

1. Copy the relevant ZIP file from its folder:
    - English/raw.zip, or
    - Hebrew/raw_no_scrolls.zip
2. Place the selected file in the **root directory** of the runtime environment

3. Rename it to:
4. raw.zip

Do **not** unzip the file manually.
Extraction is handled entirely by the system during execution.

4. Selecting the Analysis Dataset (Required Manual Configuration)

The system supports two datasets:

- **Shakespeare (English)**
- **Tanach / Hebrew Bible**

Because these datasets require different preprocessing and evaluation settings, the user must **manually align predefined configuration blocks** before execution.

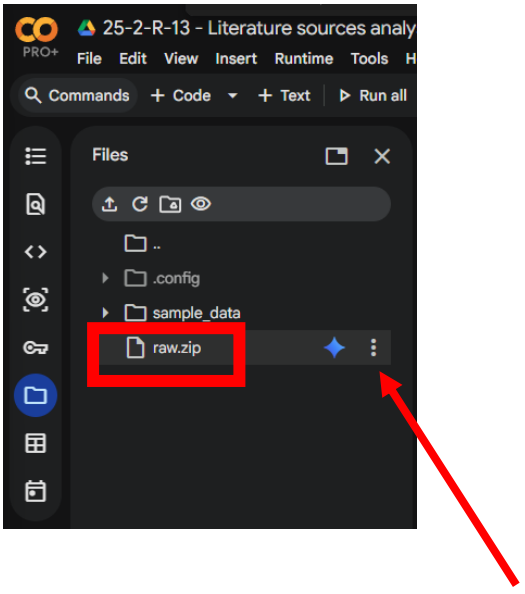
All required changes are grouped in clearly labeled cells in the notebook.

4.1 Shakespeare Dataset Selection

To analyze the **Shakespeare** dataset, complete **all** of the following steps **before running the notebook**.

Step 1: Dataset File

- Ensure the file in the root directory is:
- raw.zip
- The file must originate from:
- English/raw.zip



Rename the selected dataset file to raw.zip in the runtime environment using the three dots.

Step 2: Target Tweaks

Under the cell titled **“Target Tweaks”**:

- Enable (uncomment) the block labeled **“For Shakespeare”**
- Disable (comment out) the block labeled **“For Tanach”**

You can click inside the cell with the mouse and then use"ctrl + A" to mark all text and then "ctrl + /" to commet or uncommet

Step 3: Evaluation Harness

Under the cell titled **“Evaluation Harness”**:

- Enable the version of **build_smart_ground_truth** labeled **“For Shakespeare”**
- **.**Disable the version labeled **“For Tanach”**

```python
# for shakespear
def build_smart_ground_truth(doc_ends, data_dir, outliers, total_len=None):
    """
    Ground-truth boundary INDICES (global chunk indices).
    Marks ONLY the chunk index where a suspect/outlier text STARTS (stable → outlier).
    """
    src_path = Path(data_dir) / "targets"

    files = sorted(
        [f.name for f in src_path.glob("*.txt") if not f.name.startswith(".")],
        key=str.lower
    )

    if len(files) != len(doc_ends):
        raise ValueError(
            f"Mismatch: {len(files)} target files but {len(doc_ends)} doc_ends. "
            "This means your file ordering / filtering differs between GT and pipeline."
        )

    # safest default: total number of chunks in concatenated stream
    if total_len is None:
        total_len = int(doc_ends[-1])

    outliers_norm = {normalize_name(x) for x in outliers}
    gt_indices = []

    for i in range(len(files) - 1):
        curr_is_out = normalize_name(files[i]) in outliers_norm
        next_is_out = normalize_name(files[i + 1]) in outliers_norm

        # ONLY mark the START of an outlier (stable → outlier)
        if (not curr_is_out) and next_is_out:
            boundary_idx = int(doc_ends[i])  # start chunk of next doc (the outlier)
            if 0 <= boundary_idx < int(total_len):
                gt_indices.append(boundary_idx)

    return gt_indices
```

```python
# # For tanach
# def build_smart_ground_truth(doc_ends, data_dir, outliers, total_len=None):
#     """
#     Ground-truth boundary INDICES (global chunk indices) for ALL document boundaries.

#     If you concatenated targets in order:
#       target0 | target1 | target2 | ... | targetN-1

#     then doc_ends[i] is the chunk index where:
#       target_i ends AND target_{i+1} starts.

#     So GT boundaries = doc_ends[:-1].
#     """
#     if doc_ends is None or len(doc_ends) < 2:
#         return []

#     # safest default: total number of chunks in concatenated stream
#     if total_len is None:
#         total_len = int(doc_ends[-1])

#     gt_indices = []
#     for i in range(len(doc_ends) - 1):
#         boundary_idx = int(doc_ends[i])
#         if 0 <= boundary_idx < int(total_len):
#             gt_indices.append(boundary_idx)

#     return gt_indices
```

Step 4: Post-Training Configuration

Under the cell titled **“Post Training – Tweak Configuration”**:

- Enable _cfg_custom_lowVote labeled **“For Shakespeare”**
- Disable _cfg_custom_lowVote labeled **“For Tanach”**

Repeat **Step 4** in the cell titled **“Non-Maximum Suppression”**.

```
# For ShakeSpeare
_cfg_custom_lowVote = with_overrides(
    _custom_cfg,
    window_w=4,
    wavelet_family="haar",
    wavelet_level=4,
    vote_threshold=0.31,
    min_peak_distance=10,
    peak_height = None,
    peak_prominence=0.03,
    eval_tol=50
)

# # For Tanach
# # 1. Tweak Vote Threshold (Configuration)
# _cfg_custom_lowVote = with_overrides(
#     _custom_cfg,
#     window_w=2,
#     wavelet_family="haar",
#     wavelet_level=4,
#     vote_threshold=0.30,
#     min_peak_distance=2,
#     peak_height = None,
#     peak_prominence=0.015,
#     eval_tol=5
# )
```

Completion Check

Before execution, verify that:

- All enabled blocks correspond to **Shakespeare**
- All Tanach-related blocks are disabled
- Only one dataset configuration is active

After completing these steps, the system is correctly configured for **Shakespeare analysis**.

4.2 Tanach Dataset Selection

To analyze the **Tanach (Bible)** dataset, complete **all** of the following steps **before running the notebook**.

Step 1: Dataset File

- Ensure the file in the root directory is:
- raw.zip
- The file must originate from:
- Hebrew/raw_no_scrolls.zip

Step 2: Target Tweaks

Under the cell titled **“Target Tweaks”**:

- Enable (uncomment) the block labeled **“For Tanach”**
- Disable (comment out) the block labeled **“For Shakespeare”**

You can click inside the cell with the mouse and then use"ctrl + A" to mark all text and then "ctrl + /" to commet or uncommet

Step 3: Evaluation Harness

Under the cell titled **“Evaluation Harness”**:

- Enable the version of **build_smart_ground_truth** labeled **“For Tanach”**
- Disable the version labeled **“For Shakespeare”**

```python
# For tanach
def build_smart_ground_truth(doc_ends, data_dir, outliers, total_len=None):
    """
    Ground-truth boundary INDICES (global chunk indices) for ALL document boundaries.

    If you concatenated targets in order:
      target0 | target1 | target2 | ... | targetN-1

    then doc_ends[i] is the chunk index where:
      target_i ends AND target_{i+1} starts.

    So GT boundaries = doc_ends[:-1].
    """
    if doc_ends is None or len(doc_ends) < 2:
        return []

    # safest default: total number of chunks in concatenated stream
    if total_len is None:
        total_len = int(doc_ends[-1])

    gt_indices = []
    for i in range(len(doc_ends) - 1):
        boundary_idx = int(doc_ends[i])
        if 0 <= boundary_idx < int(total_len):
            gt_indices.append(boundary_idx)

    return gt_indices
```

```python
# # for shakespear
# def build_smart_ground_truth(doc_ends, data_dir, outliers, total_len=None):
#     """
#     Ground-truth boundary INDICES (global chunk indices).
#     Marks ONLY the chunk index where a suspect/outlier text STARTS (stable → outlier).
#     """
#     src_path = Path(data_dir) / "targets"

#     files = sorted(
#         [f.name for f in src_path.glob("*.txt") if not f.name.startswith(".")],
#         key=str.lower
#     )

#     if len(files) != len(doc_ends):
#         raise ValueError(
#             f"Mismatch: {len(files)} target files but {len(doc_ends)} doc_ends. "
#             "This means your file ordering / filtering differs between GT and pipeline."
#         )

#     # safest default: total number of chunks in concatenated stream
#     if total_len is None:
#         total_len = int(doc_ends[-1])

#     outliers_norm = {normalize_name(x) for x in outliers}
#     gt_indices = []

#     for i in range(len(files) - 1):
#         curr_is_out = normalize_name(files[i]) in outliers_norm
#         next_is_out = normalize_name(files[i + 1]) in outliers_norm

#         # ONLY mark the START of an outlier (stable → outlier)
#         if (not curr_is_out) and next_is_out:
#             boundary_idx = int(doc_ends[i])  # start chunk of next doc (the outlier)
#             if 0 <= boundary_idx < int(total_len):
#                 gt_indices.append(boundary_idx)

#     return gt_indices
```

Step 4: Post-Training Configuration

Under the cell titled **"Post Training – Tweak Configuration"**:

- Enable _cfg_custom_lowVote labeled **"For Tanach"**
- Disable _cfg_custom_lowVote labeled **"For Shakespeare"**

Repeat **Step 4** in the cell titled **"Non-Maximum Suppression"**.

```
# 1. Tweak Vote Threshold (Configuration)

# # For ShakeSpeare
# _cfg_custom_lowVote = with_overrides(
#     _custom_cfg,
#     window_w=4,
#     wavelet_family="haar",
#     wavelet_level=4,
#     vote_threshold=0.31,
#     min_peak_distance=10,
#     peak_height = None,
#     peak_prominence=0.03,
#     eval_tol=50
# )

# For Tanach
# 1. Tweak Vote Threshold (Configuration)
_cfg_custom_lowVote = with_overrides(
    _custom_cfg,
    window_w=2,
    wavelet_family="haar",
    wavelet_level=4,
    vote_threshold=0.30,
    min_peak_distance=2,
    peak_height = None,
    peak_prominence=0.015,
    eval_tol=5
)
```

Completion Check

Before execution, verify that:

- All enabled blocks correspond to **Tanach**
- All Shakespeare-related blocks are disabled
- Only one dataset configuration is active

After completing these steps, the system is correctly configured for **Tanach analysis**.

5. Running the System

Once **either Section 4.1 or 4.2** is completed:

Runtime → Run all

No further manual interaction is required.

6. First Run, Re-Runs, and Parameter Exploration

6.1 First Execution (Initial Model Training)

When running the system **for the first time**, the neural models must be **trained from scratch**.

During this initial execution, the system will:

- Train the models
- Generate stylistic signals
- Cache intermediate results for reuse

This process may take a **significant amount of time**, depending on the dataset size and available hardware.

The first execution should be performed using the **default cached execution command**, which builds and stores the required artifacts automatically.

6.2 Post-Training Parameter Exploration (No Retraining)

After the first execution completes, the trained models and generated signals are stored on disk.

At this stage, the user may safely experiment with detection behavior by adjusting parameters in the following cells:

- **Post Training – Tweak Configuration**
- **Non-Maximum Suppression**

These adjustments:

- Do **not** retrain the neural models
- Only affect how stylistic boundaries are detected and filtered
- Allow the user to observe how the system reacts to different sensitivity settings

This enables fast experimentation without repeating the full training process.

6.3 Retraining the Models (Optional)

If the user wishes to **retrain the models** (for example, after changing datasets or core settings), this can be done by modifying the execution mode in the cell titled:

“How to Run the Orchestrators – Cached pipeline (single backend, faster re-runs)”

Specifically:

- Set use_cache to **False**
- This forces the system to rebuild all models and cached artifacts during the next execution

From:

```
# 1) Run full First run (build & cache)
res_cached_custom_cfg = run_pipeline_cached(_custom_cfg, use_cache=True)
```

TO:

```
# 1) Run full First run (build & cache)
res_cached_custom_cfg = run_pipeline_cached(_custom_cfg, use_cache=False)
```

After this change, the notebook should again be executed using:

Runtime → Run all

Important Usage Rule

- **First run** → model training is required
- **Subsequent runs** → parameters can be tuned without retraining
- **Retraining** → only required when explicitly requested by the user

Maintenance Guide

1. Operating Environment

To ensure the continued operation, reproducibility, and stability of the **Deep Impostor** system, the following hardware and software infrastructure is required.

Hardware Requirements

- **Processing Unit**:
  A GPU (Graphics Processing Unit) is **mandatory** for training the BERT models and BiLSTM networks within reasonable time and memory limits.
  - **Recommended**: NVIDIA T4 Tensor Core (standard in Google Colab Free Tier) or stronger (e.g., V100 / A100).
  - **Minimum VRAM**: 12 GB
    This is required to support batch sizes in the range of 16–32 without encountering out-of-memory (OOM) errors.
- **Storage**:
  At least 2 GB of free disk space for:
  - dataset extraction
  - cached artifacts (model checkpoints, signal matrices, embeddings)
  - exported results

Software Infrastructure

- **Platform**: Google Colab (Jupyter Notebook environment).
- **Runtime Environment**: Python 3.10 or newer.
- **Key Dependencies**:
  The system relies on several Python libraries that are required for model training, signal processing, and evaluation:
  - torch (PyTorch) – used by Transformer-based models
  - transformers (Hugging Face) – BERT and related architectures
  - gensim – Word2Vec implementation
  - pywt (PyWavelets) – wavelet-based signal processing
  - numpy, pandas, scikit-learn – data manipulation and evaluation

Note:
The system uses both **TensorFlow/Keras** and **PyTorch-based** components, depending on the selected backend. This coexistence is intentional and handled internally by the notebook.

2. Installation and Deployment Instructions

Since the system operates in a managed notebook environment, “installation” refers to preparing the project files and execution context rather than deploying a standalone application.

2.1 Project Initialization

- Load the notebook file
  25_2_R_13_Literature_sources_analysis_using_deep_impostor
  _approach.ipynb
  into the Google Colab environment.
- Set the runtime type to **GPU** (Runtime → Change runtime type → GPU).

2.2 Data Structure Setup

- The system includes automated data ingestion logic.
- The maintainer does **not** need to manually create directories.

Action:

- Upload a dataset ZIP file named raw.zip to the root directory.

During the data ingestion stage, the system automatically creates the following directory structure:

- data/targets/ – extracted target texts
- data/impostors/ – extracted impostor texts
- artifacts/ – cached model outputs and intermediate matrices
- results/ – exported evaluation results and visualizations

3. Dependency Installation

- Execute the **first notebook cell**.
- This cell installs all required Python packages using pip, including libraries that are not pre-installed in the default Colab environment (e.g., ebooklib, gensim, pywt).

No manual dependency management is required beyond this step.

4. Routine Maintenance and Updates

This section describes how to maintain, adapt, and extend the system over time.

4.1 Updating Configuration Parameters

- All core operational parameters are centralized in:
  - the Config dataclass
  - the _custom_cfg configuration object
- **Sensitivity tuning**:
  - Adjust vote_threshold or window_w in the **Post Training – Tweak Configuration** stage.
  - These changes affect detection behavior without retraining models.
- **Resource adaptation**:
  - If hardware constraints change (e.g., weaker GPU), reduce the batch_size parameter in _custom_cfg (e.g., from 64 to 16).

No structural code changes are required for these updates.

4.2 Adding New Datasets

To extend the system to a new corpus (e.g., historical or domain-specific documents):

- Create a new ZIP archive (raw_<name>.zip) containing the text files.
- Upload it to the runtime environment as raw.zip.

Code updates required:

- Update the CURRENT_OUTLIERS list in the **Target Tweaks** section to reflect the naming and structure of the new dataset.

Important:
Adding a new dataset may also require adapting **ground-truth construction** and **evaluation logic** to match the corpus structure and segmentation assumptions.

4.3 Extending Model Capabilities

Switching Transformer Models

- The system relies on Hugging Face's AutoModel architecture.
- Switching between Transformer-based models (e.g., bert-base-uncased, roberta-base, dicta-il/dictabert) does **not** require structural code changes.

Action:

- Update the bert_model_name parameter in the _custom_cfg configuration block.

4.4 Backup and Artifact Management

- The artifacts/ directory contains outputs from computationally expensive stages, including:
  - stylistic signal matrix (S)
  - distance matrices (D, R)
  - cached classifier outputs
- These artifacts enable **cached execution**, allowing rapid re-runs without retraining.

Recommendation:

- Periodically download and back up the artifacts/ directory to local storage.
- Restoring this directory allows future executions to reuse cached results and significantly reduce runtime.